

ИСКЛЮЧЕНИЯ В C#:

- Исключение (exception) - это объект, который представляет ошибку или неожиданное поведение во время выполнения программы.
- В C# исключения являются механизмом обработки ошибок и позволяют программисту предусмотреть ситуации, в которых программа может перейти в неопределенное состояние.
- Исключения могут быть сгенерированы как встроенными классами .NET, так и пользовательскими классами исключений.

ОБРАБОТКА ИСКЛЮЧЕНИЙ

- Обработка исключений позволяет программисту предусмотреть возможные ошибки и выполнить действия для их обработки.
- Блок try-catch используется для обработки исключений. Код, который может вызвать исключение помещается в блок try, а код для обработки исключения - в блок catch.
- Если исключение происходит в блоке try, управление передается блоку catch.
- В блоке catch можно выполнить необходимые действия по обработке исключения, например, вывести сообщение об ошибке или выполнить альтернативную логику.

СОЗДАНИЕ СОБСТВЕННЫХ ИСКЛЮЧЕНИЙ

- В C# можно создавать собственные классы исключений, которые наследуются от базового класса Exception.
- Создание собственного класса исключений позволяет определить собственную логику обработки ошибок и предоставить дополнительную информацию о возникшей проблеме.
- Класс пользовательского исключения должен иметь конструктор, который наследуется от базового класса и может добавлять дополнительные свойства или методы для предоставления информации об ошибке.

ГЕНЕРАЦИЯ ИСКЛЮЧЕНИЙ

- Чтобы сгенерировать исключение в коде, можно использовать оператор `throw` с исключением в блоке `try`.
- Исключение может быть сгенерировано явно, когда программа обнаруживает ошибку или неожиданное условие, или автоматически, когда возникает ошибка во время выполнения программы (например, деление на ноль).

ПРИМЕР СОБСТВЕННОГО КЛАССА ИСКЛЮЧЕНИЙ

```
public class CustomException : Exception // класс собственного исключения
{
    // конструктор наследуется от родительского класса
    public CustomException() : base("Custom exception occurred")
    {
    }
}
public class Program
{
    public static void Main(string[] args)
    {
        // в блоке try вызываем метод деления
        try
        {
            int result = Divide(10, 0);
            Console.WriteLine("Result: " + result);
        }
        // в первом блоке catch вызываем наш класс исключения
        catch (CustomException ex)
        {
            Console.WriteLine("Custom Exception: " + ex.Message);
        }
        // во втором блоке catch вызываем класс исключения .NET
        catch (Exception ex)
        {
            Console.WriteLine("Unhandled Exception: " + ex.Message);
        }
    }
    // статичный метод деления
    public static int Divide(int a, int b)
    {
        if (b == 0) // если второе число = 0, генерируем исключение
        {
            throw new CustomException();
        }
        return a / b; // иначе выполняем деление
    }
}
```

ПРАКТИКА

Практические задания для закрепления материала.

- Разработать класс `BankAccount`, который представляет собой банковский счет. В данном классе должно быть поле для хранения баланса счета и конструктор класса.

Ваша задача:

1. Создать класс `InsufficientFundsException`, который будет представлять исключение "Недостаточно средств на счете". Данный класс должен быть производным от класса `Exception`.
2. В методе `Withdraw` класса `BankAccount` обрабатывать исключения следующим образом:
 - Если на счете недостаточно средств, выбрасывать исключение `InsufficientFundsException`.
 - Если попытка снять отрицательную сумму, выбрасывать исключение `ArgumentException`.
 - Если снятие денег прошло успешно, выводить сообщение о снятии указанной суммы.
- Разработать класс `TemperatureConverter`, который предназначен для конвертации температуры между различными единицами измерения: градусами Цельсия (C), градусами Фаренгейта (F) и Кельвином (K).

Ваша задача:

1. Создать класс `InvalidTemperatureException`, который будет представлять исключение "Неверное значение температуры". Данный класс должен быть производным от класса `Exception`.
2. Реализовать метод `ConvertTemperature`, который будет принимать значения температуры и единицы измерения, а затем конвертировать температуру в другие единицы измерения.
3. Обработать исключения в методе `ConvertTemperature` следующим образом:
 - Если передана неверная единица измерения, выбрасывать исключение `InvalidTemperatureException`.
 - Если передано некорректное значение температуры, выбрасывать исключение `ArgumentException`.
 - В противном случае, выполнять конвертацию и возвращать результат.